

**ДЕРЖАВНИЙ УНІВЕРСИТЕТ
ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ
ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ**

КУРСОВА РОБОТА

на тему:

«Розробка веб-API для системи управління фітнес клубами»

з дисципліни:

«Об'єктно-орієнтоване програмування C#»

студента 2 курсу групи ПД-23

Кафедри інженерії програмного забезпечення

Ярошевського Олександра Вікторовича

Викладач

асистент кафедри

Інженерії програмного забезпечення,

Ярошевський Олександр Вікторович

оцінка _____

Київ 2026

РЕФЕРАТ

Курсова робота, 53 с., 12 рис., 3 додатки, 15 джерел.

Ключові слова: .NET CORE 3.1, ENTITY FRAMEWORK CORE, SQLITE, REST WEBAPI, CQRS, MEDIATR, AUTOFAC, SWAGGER, UNIT OF WORK, REPOSITORY PATTERN, ХЕНДЛЕР, ІНВЕРСІЯ КЕРУВАННЯ (ІОС).

Об'єктом дослідження було: процес автоматизації обліку відвідувачів, абонементів, розкладу занять та філій у сучасній мережі фітнес-клубів.

Метою дослідження було: розробка швидкого, незалежного від важких серверів СУБД, кросплатформного RESTful API з високим рівнем тестованості бізнес-логіки (BLL) за допомогою механізму тестування SQLite в оперативній пам'яті (In-Memory).

В ході дослідження були одержані: архітектурні підходи до побудови масштабованих та тестованих вебслужб на базі платформи .NET Core з розділення відповідальності на шари (Layered/Clean Architecture).

ЗМІСТ

ВСТУП	6
1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПРОЕКТУВАННЯ АРХІТЕКТУРИ	8
1.1. Опис бізнес-процесів фітнес-клубу	8
1.2. Обґрунтування вибору технологічного стеку (.NET Core, SQLite)	10
1.3. Схема бази даних та зв'язків між сутностями	12
2. ПРЕДМЕТНА РЕАЛІЗАЦІЯ СЕРВЕРНОЇ ЧАСТИНИ	15
2.1. Організація структури рішення та шарів архітектури	15
2.2. Конфігурація доступу до даних за допомогою EF Core та SQLite	18
2.3. Реалізація шаблонів Repository, Unit of Work та медіатора MediatR	20
3. ПРОЕКТУВАННЯ СТРУКТУРИ ТА АРХІТЕКТУРИ ЗАСОБАМИ UML	22
3.1. Обґрунтування вибору уніфікованої мови моделювання UML	22
3.2. Діаграма прецедентів використання (Use Case Diagram)	23
4. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ДЕМОНСТРАЦІЯ РОБОТИ СИСТЕМИ	25
4.1. Логіка організації архітектури та взаємодії компонентів системи	25
4.2. Логіка та алгоритм функціонування основного бізнес-сценарію	26
4.3. Демонстрація результатів розробки та аналіз візуальних інтерфейсів	27
4.4. Візуалізація структурної організації та шарів рішення	27
4.5. Головний графічний інтерфейс документації RESTful API (Swagger UI) ..	30
4.6. Демонстрація успішного виконання операції створення сутності	31
4.7. Демонстрація пошуку.....	32
5. ТЕСТУВАННЯ РОЗРОБЛЕНОГО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	32
5.1. Види тестувань, що використовувалися	33
5.2. Результати тестування	33
5.3. Відповідність розробленого ПЗ поставленим вимогам	36
6. Перспективи розвитку та пропозиції щодо frontend-частини системи.....	37
6.1 Подальший розвиток проекту.....	37
ВИСНОВКИ	38
ПЕРЕЛІК ПОСИЛАНЬ	40
ДОДАТКИ	42
ДОДАТОК А. Демонстраційні матеріали	42
ДОДАТОК Б. Репозиторій програмних модулів	49
ДОДАТОК В. Лістинги програмних модулів	49

СЛОВНИК ТЕРМІНІВ

Автомапер (AutoMapper) — бібліотека для автоматичного мапінгу (перетворення) об'єктів одного типу в об'єкти іншого типу, що значно спрощує роботу з DTO-моделями.

BLL (Business Logic Layer) — шар бізнес-логіки, який містить основні правила та операції системи.

Clean Architecture — архітектурний стиль, що передбачає чітке розділення шарів системи з незалежністю бізнес-логіки від зовнішніх деталей (фреймворків, баз даних тощо).

CQRS (Command Query Responsibility Segregation) — патерн проектування, який розділяє операції, що змінюють стан системи (Commands), від операцій, що лише читають дані (Queries).

DTO (Data Transfer Object) — об'єкт передачі даних, який використовується для обміну інформацією між шарами системи (наприклад, між API та клієнтом).

Entity Framework Core (EF Core) — сучасний ORM-фреймворк для .NET, що забезпечує взаємодію з базою даних через об'єктно-орієнтований підхід.

MediatR — бібліотека, що реалізує шаблон «Медіатор», використовується для обробки команд і запитів у CQRS-архітектурі.

Repository Pattern — патерн, який інкапсулює логіку доступу до даних, приховуючи деталі реалізації бази даних від бізнес-логіки.

RESTful API — архітектурний стиль побудови веб-сервісів, що базується на стандартах HTTP та принципах REST.

Swagger UI — інструмент для автоматичної генерації інтерактивної документації REST API.

Unit of Work (UoW) — патерн, який керує транзакціями та забезпечує узгодженість даних при виконанні кількох операцій з базою даних в одній логічній операції.

Web API — програмний інтерфейс додатка, доступний через мережу (HTTP), призначений для обміну даними між клієнтом і сервером.

Автентифікація — процес перевірки ідентичності користувача.

Абонемент (ClubPass) — основний комерційний продукт фітнес-клубу, що надає право відвідування занять протягом певного періоду або кількості разів.

Клуб (Club) — територіальна одиниця мережі фітнес-центрів, що має власну назву, адресу та набір послуг.

Користувач (User) — клієнт системи, який може мати активний абонемент та/або запис на тренування.

Тренування (Workout) — заплановане заняття у фітнес-клубі з певним типом, часом проведення та вартістю.

In-Memory Database — база даних, що зберігається в оперативній пам'яті, використовується для швидкого юніт- та інтеграційного тестування.

Іверсія керування (IoC) — принцип, згідно з яким керування залежностями об'єктів передається зовнішньому контейнеру (наприклад, Autofac).

ВСТУП

Актуальність теми. Сучасний етап розвитку світової індустрії спорту та здоров'я характеризується стрімким зростанням попиту на послуги фітнес-центрів, що, у свою чергу, провокує загострення конкуренції між компаніями та змушує менеджмент шукати нові шляхи оптимізації внутрішньої діяльності. Ефективне функціонування сучасної мережі фітнес-клубів неможливе без повної цифровізації її бізнес-процесів. Автоматизація обліку клієнтської бази, контроль термінів дії різноманітних тарифних планів (абонементів), координація розкладу групових та індивідуальних занять, а також аналітика роботи окремих територіальних філій вимагають створення надійного, централізованого та швидкодіючого програмного забезпечення. Впровадження таких систем дозволяє повністю нівелювати вплив людського фактора, мінімізувати черги на рецепції, раціонально розподіляти навантаження на спортивні зали та забезпечити високу швидкість обробки HTTP-запитів користувачів у реальному часі.

З погляду інженерії програмного забезпечення, проектування сучасних закритих або публічних вебслужб висуває жорсткі вимоги до їхньої архітектурної витонченості. Замість класичних монолітних підходів, які ведуть до високої зв'язаності компонентів (Tight Coupling) та ускладнюють подальший супровід коду, на перший план виходить побудова систем на основі розподілу відповідальності (шарів) — так звана Clean/Layered Architecture. Окрім цього, критично важливим аспектом є забезпечення високого рівня тестованості бізнес-логіки системи. Традиційні методи тестування, які спираються на фейкові об'єкти або важкі зовнішні СУБД, часто є або занадто поверхневими, або надто ресурсомісткими. Тому дослідження та практична реалізація архітектурних рішень із використанням легких, вбудованих реляційних баз даних (як-от SQLite) для забезпечення ізольованого тестування є надзвичайно актуальним прикладним завданням.

Метою даної роботи є розробка кросплатформного, швидкого та незалежного від важких серверних СУБД сервісу RESTful WebAPI для автоматизації управління мережею фітнес-клубів, із реалізацією розділення потоків даних (CQRS), інверсії

керування (IoC) та забезпеченням ізольованого інтеграційного тестування бізнес-моделей.

1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПРОЕКТУВАННЯ АРХІТЕКТУРИ

1.1 Опис бізнес-процесів фітнес-клубу

Сучасний фітнес-центр або розгалужена мережа спортивних комплексів є складним конгломератом взаємопов'язаних операційних процесів, які вимагають безперервного контролю та точного обліку даних. Головна мета автоматизації цієї предметної області полягає у зведенні до мінімуму рутинних операцій адміністраторів, виключенні помилок, пов'язаних із людським фактором, та забезпеченні прозорості у взаємодії між клієнтом і клубом. У рамках даного проекту було проведено детальний системний аналіз діяльності фітнес-мережі, в результаті якого виділено чотири фундаментальні бізнес-сутності, що складають ядро системи.

Першою базовою сутністю є **Фітнес-клуб (Club)**. У масштабах мережі кожна філія функціонує як окрема територіальна одиниця, що має власні унікальні характеристики: комерційну назву, географічне розташування (місто) та точну юридичну чи фактичну адресу. Облік клубів дозволяє системі диференціювати потоки відвідувачів та прив'язувати матеріально-технічні ресурси (зали, розклад, тренерський склад) до конкретних локацій.

Другою критично важливою сутністю є **Абонемент (ClubPass)**, який виступає основним комерційним продуктом фітнес-мережі та головним інструментом регулювання прав доступу клієнтів. Абонемент не є просто записом про оплату; це складна бізнес-модель тарифного плану. Він характеризується унікальною назвою (наприклад, «Студентський ранок», «Річний безліміт»), вартістю, тривалістю дії, що обчислюється у календарних днях, та лімітом доступних занять (кількістю візитів). Ключовим бізнес-правилом, реалізованим у системі, є поділ абонементів за типом покриття за допомогою логічного прапорця IsNet (Мережевий). Якщо цей параметр має значення true, абонемент вважається глобальним мережевим продуктом, що дає право клієнту відвідувати будь-яку філію в межах країни. Якщо ж isNet дорівнює false, система жорстко обмежує валідацію абонементу лише у межах тієї конкретної

філії (ClubId), де він був безпосередньо придбаний.

Третя сутність — **Клієнт (User)** — репрезентує кінцевого споживача послуг. Облік користувачів передбачає збереження їхніх персональних даних, таких як повне ім'я та прізвище. Особливістю проектування цієї сутності є гнучкість взаємодії з іншими об'єктами. Під час первинної реєстрації відвідувача в базі даних архітектура системи дозволяє реалізувати гнучкий життєвий цикл: користувача можна створити як «гостя» (без активного абонементу та без запису на тренування), або ж одразу в момент реєстрації виконати комплексний бізнес-процес — закріпити за ним придбаний ClubPassId та автоматично записати на конкретне заняття.

Четверта сутність — **Тренування (Workout)** — відображає динамічний розклад діяльності залів. Кожне тренування описує конкретну спортивну подію (груповий кросфіт, стретчинг, персональне заняття з тренером) і містить інформацію про напрямок (тип), точний час проведення у міжнародному форматі UTC, вартість одиничного відвідування та обов'язкову прив'язку до локації (ClubId), де воно фізично відбудеться.

1.2 Обґрунтування вибору технологічного стеку (.NET Core 3.1, SQLite)

Створення сучасного програмного забезпечення корпоративного рівня вимагає ретельного підбору інструментів розробки, які здатні забезпечити високу швидкість обробки даних, безпеку типів, легкість розгортання та кросплатформність. На основі цих критеріїв для реалізації серверної частини системи було обрано інженерну платформу .NET Core 3.1 від корпорації Microsoft. Ця версія платформи характеризується довготривалим циклом підтримки, оптимізованим рушієм виконання CoreCLR та модульною структурою. Використання мови програмування C# з її строгою типізацією дозволяє виявляти значну частину архітектурних та логічних помилок ще на етапі компіляції проєкту.

Важливим кроком проектування стало архітектурне рішення щодо вибору системи керування базами даних (СУБД). У традиційних проєктах прийнято використовувати важкі клієнт-серверні СУБД, такі як MS SQL Server, PostgreSQL або MySQL. Проте, для завдань локального тестування, демонстрації працездатності концептів (Proof of Concept) та захисту курсових робіт розгортання повноцінного сервера бази даних супроводжується низкою проблем: високими вимогами до апаратних ресурсів ЕОМ, необхідністю попереднього ліцензування, складним адмініструванням та конфігурацією портів і прав доступу.

З огляду на це, як сховище даних було обрано вбудовану (embedded) СУБД SQLite у поєднанні з передовою технологією об'єктно-реляційного відображення (ORM) Entity Framework Core. SQLite кардинально відрізняється від класичних СУБД тим, що її рушій інтегрується безпосередньо всередину скомпільованого коду програми, а вся реляційна база даних разом зі схемами, індексами та тригерами фізично записується в один-єдиний локальний файл на диску (FitnessContextDb.db). Такий підхід забезпечує унікальні переваги:

1. *Повна автономність*: Проєкт стає максимально мобільним і переносним.

Програму можна скопіювати на будь-який інший комп'ютер, і вона запуститься без додаткових маніпуляцій чи встановлення стороннього софту.

2. *Кросплатформність*: Локальний файл бази даних однаково успішно зчитується та модифікується в операційних системах Windows, Linux та macOS.
3. *Продуктивність*: Оскільки операції читання та запису відбуваються через прямий доступ до локального файлового потоку без мережових затримок (network overhead), які притаманні клієнт-серверним архітектурам, швидкість виконання простих транзакцій є надзвичайно високою. При цьому Entity Framework Core дозволяє абстрагуватися від написання сирого SQL-коду, автоматично трансліюючи LINQ-запити C# у високооптимізовані SQL-інструкції, специфічні для діалекту SQLite, повністю зберігаючи принципи транзакційності ACID.

1.3 Схема бази даних та зв'язків між сутностями

Проектування схеми даних базувалося на принципах нормалізації реляційних баз даних для уникнення аномалій вставки, оновлення та видалення інформації. Структура створеної бази даних побудована на базі суворих логічних зв'язків типу «один-до-багатьох» (\$1:N\$), які забезпечують реляційну цілісність інформаційного простору фітнес-клубу.

Головним вузлом, навколо якого групуються інші дані, виступає таблиця Клубів (Clubs). Зв'язок між таблицею Clubs та таблицею Абонементів (ClubPasses) реалізовано як класичний \$1:N\$. Один конкретний фітнес-клуб може випускати та обслуговувати необмежену кількість тарифних планів та карт, проте кожен конкретний локальний абонемент містить обов'язкове поле зовнішнього ключа ClubId, яке вказує на його батьківський зал. Аналогічний зв'язок \$1:N\$ спроектовано між таблицями Clubs та Тренуваннями (Workouts). Одна філія мережі формує свій унікальний щоденний розклад, що складається з багатьох занять, але кожне окреме тренування може проходити тільки в одному чітко визначеному клубі, що фіксується за допомогою непустиго зовнішнього ключа ClubId.

Найбільш цікавим та гнучким з інженерного погляду є проектування таблиці Користувачів (Users). Для забезпечення бізнес-процесів розробки, сутність User містить два зовнішні ключі: ClubPassId (зв'язок із таблицею абонементів) та WorkoutId (зв'язок із таблицею тренувань). Обидва ці поля на рівні схеми SQLite оголошені як **Nullable** (такі, що можуть приймати значення NULL).

Це архітектурне рішення дозволяє системі підтримувати три різні стани життєвого циклу клієнта:

1. *Чиста реєстрація*: Створення облікового запису нової людини, яка ще не визначилася з покупкою послуг (обидва ключі дорівнюють null).
2. *Придбання абонементу*: Користувач купує картку, і поле ClubPassId заповнюється відповідним ідентифікатором, надаючи йому право доступу до залів.
3. *Активний запис на заняття*: Користувач записується на групову програму, і

поле `WorkoutId` починає посилатися на конкретне тренування в розкладі, що дозволяє хендлерам бізнес-логіки контролювати ліміти завантаженості залів.

Каскадне видалення об'єктів налаштовано таким чином, щоб при видаленні тренування чи абонементу пов'язані користувачі не видалялися фізично, а лише скидали свої зовнішні посилання в `null`, що гарантує збереження клієнтської бази.

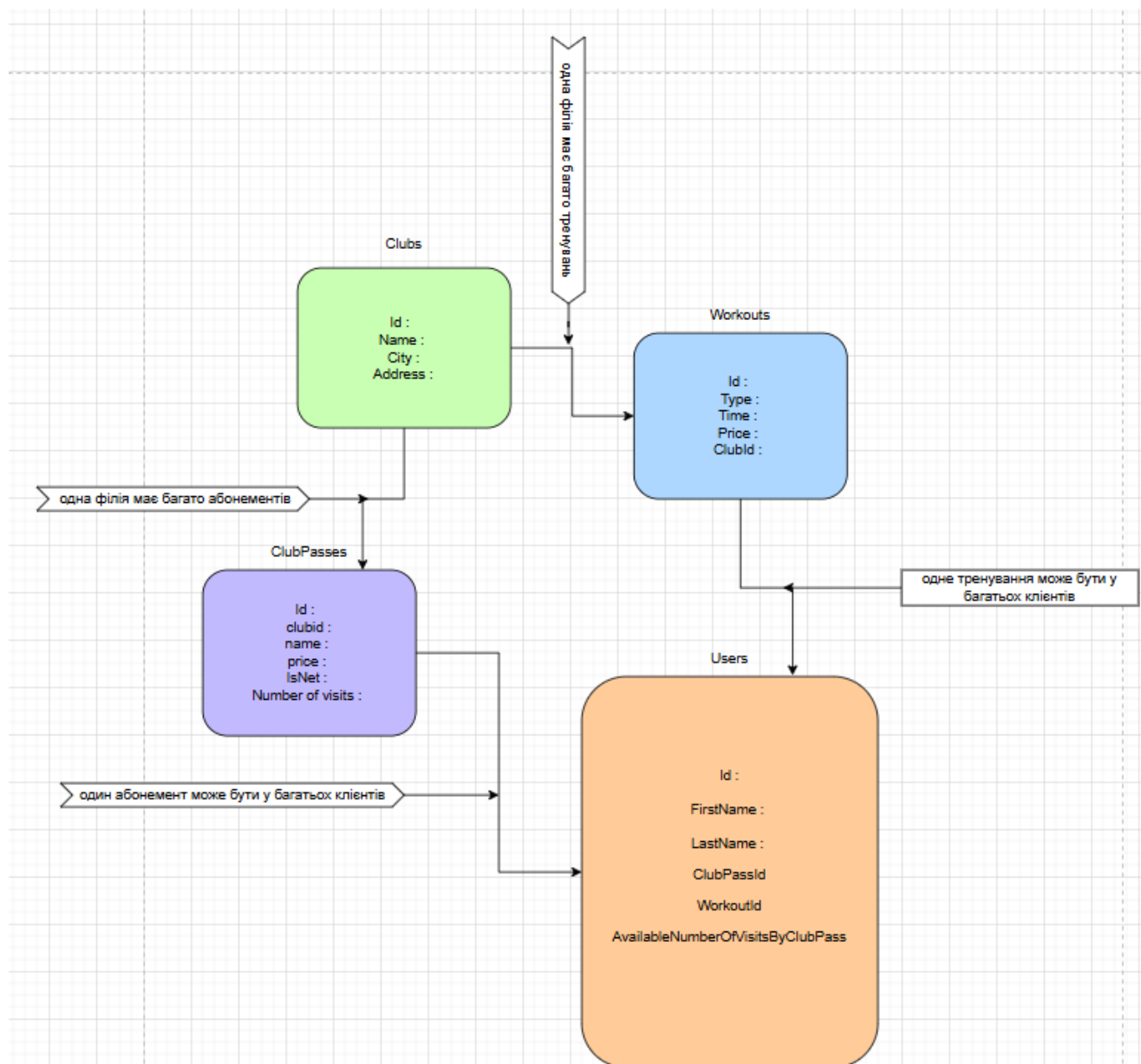


Рисунок 1.3 - Схема реляційної моделі даних інформаційної системи фітнес-клубу

2. ПРЕДМЕТНА РЕАЛІЗАЦІЯ СЕРВЕРНОЇ ЧАСТИНИ

2.1. Організація структури рішень та шарів архітектури (UI, DataBase, BLL, UoW)

Під час розробки серверної архітектури проєкту Fitness-Club-WebAPI було категорично відкинуто концепцію монолітної структури, де інтерфейс, логіка та доступ до бази даних змішуються в одному проєкті. Натомість було впроваджено принципи розділення відповідальності на базі багаторівневої архітектури (Layered / Clean Architecture). Увесь вихідний код розподілено між чотирма повністю ізольованими проєктами в межах єдиного Visual Studio Solution, кожен з яких виконує свою суворо визначену роль.

1. UI.WebAPI (Шар представлення): Даний проєкт є точкою входу в застосунок. Він реалізований як веб-сервіс ASP.NET Core WebAPI. Його головна відповідальність — прийом вхідних HTTP-запитів від клієнтів (у форматі JSON), первинна маршрутизація, перевірка коректності переданих HTTP-заголовків та повернення клієнту стандартизованих відповідей зі статус-кодами (наприклад, 200 OK або 400 Bad Request). Цей шар нічого не знає про те, як саме дані зберігаються у файлі на диску, і не містить формул розрахунку бізнес-процесів. Він лише взаємодіє з бібліотекою Swagger UI для візуалізації документації та контролює життєвий цикл вебсервера Kestrel.
2. DataBase (Шар інфраструктури даних): Це ізольована бібліотека класів, яка повністю інкапсулює все, що пов'язано з реляційним сховищем. Тут оголошено головний клас контексту FitnessContext, успадкований від DbContext, а також прописано конфігурації мапінгу C#-моделей на таблиці SQLite. Жоден інший шар програми не має права писати прямі SQL-запити; усі звернення до сховища проходять через суворі інтерфейси цього проєкту.
3. CommandsAndQueries (BLL — Шар бізнес-логіки): Це інтелектуальне серце системи. Шар містить опис усіх бізнес-сценаріїв програми, розділених за концепцією CQRS на Команди (модифікація даних, наприклад

AddUserCommand, AddClubCommand) та Запити (читання даних). Тут зосереджені так звані обробники — Хендлери (Handlers). Кожен хендлер являє собою ізольований клас, який приймає вхідну команду, виконує над нею перевірки, звертається до інфраструктури збереження та повертає результат.

4. UoW / Repository (Шар абстракції доступу): Додатковий рівень, який виступає буфером між бізнес-логікою (BLL) та контекстом ORM, захищаючи програму від жорсткої прив'язки до конкретної специфіки Entity Framework Core.

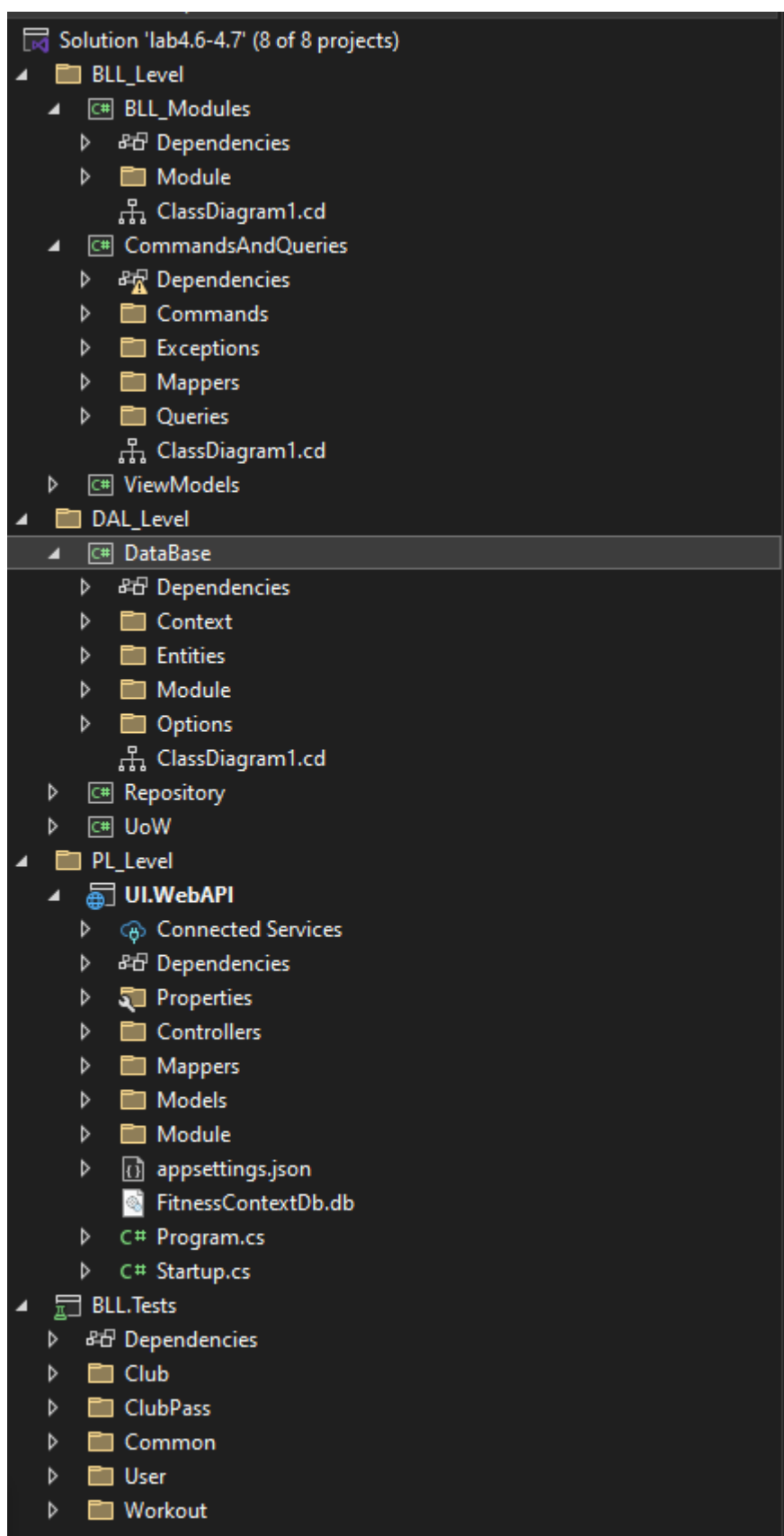


Рисунок 2.1 - Структурна організація шарів архітектури рішення у середовищі Visual Studio

2.2 Конфігурація доступу до даних за допомогою EF Core та SQLite

Управління життєвим циклом та підключенням до бази даних SQLite в межах проєкту реалізовано через патерн фабрики. Для цього в проєкті DataBase було розроблено статичний клас `DbContextOptionsFactory`. Це рішення дозволило централізувати налаштування провайдера бази даних в одному місці та уникнути дублювання конфігураційних рядків у тестовому та основному проєктах.

Фабрика містить метод `Get()`, який ініціалізує об'єкт `DbContextOptionsBuilder` для нашого контексту `FitnessContext`. Всередині цього методу викликається розширення `.UseSqlite()`, куди передається рядок підключення: `Data Source=FitnessContextDb.db`. Коли веб-додаток запускається, вебсервер звертається до цієї фабрики, і Entity Framework автоматично визначає поточну робочу директорію процесу. Файл бази даних `FitnessContextDb.db` створюється безпосередньо в папці компіляції додатка (`bin/Debug/netcoreapp3.1/`).

Для того, щоб позбавити розробника та кінцевого користувача від необхідності вручну створювати таблиці, писати SQL-скрипти створення схем або запускати складні консольні міграції, в архітектуру системи було закладено принцип автоматичної ініціалізації. Під час старту програми у файлі `Startup.cs` проєкту `UI.WebAPI` впроваджено виклик методу: `context.Database.EnsureCreated();`

Ця функція працює за таким алгоритмом: при першому HTTP-запиті до Swagger або сервера, EF Core перевіряє наявність файлу `FitnessContextDb.db` на диску. Якщо файлу немає, ORM миттєво створює його, аналізує структуру класів сутностей `Club`, `User`, `Workout`, автоматично генерує необхідні SQL-команди `CREATE TABLE` з урахуванням усіх типів даних, первинних (`PrimaryKey`) та зовнішніх (`ForeignKey`) ключів і розгортає абсолютно чисту, готову до роботи реляційну базу даних за лічені мілісекунди.

Таблиця 2.2 Компоненти архітектурних рівнів системи

Назва шару	Технологічний стек / Бібліотеки	Функціональне призначення
UI.WebAPI	.NET Core 3.1, Swagger UI	Прийом HTTP-запитів, документація ендпоінтів
CommandsAndQueries	MediatR, Autofac	Обробка бізнес-логіки за патерном CQRS
DataBase	EF Core, SQLite	Опис контексту даних, мапінг та збереження

2.3 Реалізація шаблонів Repository, Unit of Work та медіатора MediatR

Для досягнення максимальної гнучкості, розширюваності та відповідності принципам SOLID, у проєкті було відкинута ідея прямого впровадження контексту бази даних у контролери. Натомість було реалізовано тріаду передових архітектурних рішень: паттерни Repository, Unit of Work та медіатор MediatR.

Патерн Repository (Репозиторій) був створений для того, щоб інкапсулювати логіку доступу до даних. Для кожної сутності створено свій репозиторій (наприклад, ClubRepository). Замість того, щоб у бізнес-логіці писати складні конструкції на кшталт `_context.Clubs.Add(entity)`, розробник викликає простий метод `_clubRepository.Create(club)`. Це дозволяє повністю приховати специфіку роботи ORM: якщо в майбутньому ми вирішимо замінити SQLite на іншу СУБД, код бізнес-логіки в хендлерах не зміниться ні на один рядок — достатньо буде лише переписати внутрішню реалізацію методів репозиторію.

Патерн Unit of Work (Одиниця роботи) вирішує проблему узгодженості даних та управління транзакціями. Коли бізнес-сценарій вимагає виконання кількох послідовних операцій у базі даних (наприклад, створення нового користувача, списання коштів за абонемент та оновлення кількості вільних місць на тренуванні), вкрай важливо, щоб ці операції виконувалися атомарно. Unit of Work об'єднує всі репозиторії під єдиним контекстом і надає метод `SaveChanges()`. Усі зміни спочатку накопичуються в оперативній пам'яті, і лише при виклику методу збереження UoW вони записуються у файл бази даних в межах єдиної транзакції SQLite. Якщо хоча б одна операція завершиться помилкою, відбудеться повний відкат (Rollback), запобігаючи появі «битих» чи розсинхронізованих даних.

Архітектурний патерн CQRS (Command Query Responsibility Segregation) впроваджено за допомогою популярної бібліотеки MediatR. Його суть полягає у суворому розділенні операцій модифікації даних від операцій читання. Веб-контролери в шарі UI.WebAPI стали повністю «тонкими». Вони не містять жодної логіки; отримавши JSON-запит на створення, наприклад, клубу, контролер формує DTO-об'єкт команди `AddClubCommand` і відправляє його в одну єдину точку — шину

медіатора: `_mediator.Send(command)`.

MediatR за допомогою механізмів рефлексії автоматично сканує збірки програми, знаходить відповідний клас обробника `AddClubCommandHandler`, впроваджує в нього через конструктор необхідні залежності `Unit of Work` та передає команду на виконання. Обробник створює сутність, генерує унікальний ідентифікатор `Guid` і через медіатор повертає його назад у контролер, який віддає клієнту відповідь `200 OK`.

Для того, щоб уся ця складна система взаємозв'язків та інтерфейсів працювала автоматично, в проєкт було інтегровано потужний інверсійний контейнер (IoC) `Autofac`. У файлі конфігурації розробник реєструє типи: «для інтерфейсу `IUnitOfWork` завжди створювати екземпляр класу `UnitOfWork`», а також реєструє всі хендлери `MediatR`. `Autofac` бере на себе всю рутинну роботу зі збирання об'єктних графів, автоматично підставляючи потрібні сервіси у конструктори класів у момент їхнього виклику, забезпечуючи слабку зв'язаність компонентів системи.

3. ПРОЕКТУВАННЯ СТРУКТУРИ ТА АРХІТЕКТУРИ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ЗАСОБАМИ UML

3.1. Обґрунтування вибору уніфікованої мови моделювання UML для проектування систем

Процес розробки сучасних складних програмних комплексів корпоративного рівня вимагає від архітектора та інженерів програмного забезпечення попереднього побудови детальних абстрактних моделей, що описують систему з різних точок зору. Моделювання дозволяє ще до етапу безпосереднього написання вихідного коду (coding) проаналізувати повноту вимог, виявити потенційні архітектурні колізії, оптимізувати зв'язки між компонентами та зафіксувати логіку взаємодії суб'єктів із системою. У рамках даного проєкту як головний інструментарій візуального моделювання та проектування було обрано Уніфіковану мову моделювання UML (Unified Modeling Language).

UML являє собою стандартизовану графічну мову для опису, візуалізації, конструювання та документування артефактів об'єктно-орієнтованого програмного забезпечення. Застосування UML дозволяє абстрагуватися від конкретного синтаксису мови програмування C# та зосередитися на фундаментальних інженерних аспектах. У рамках розробки архітектури сервісу автоматизації фітнес-клубів було застосовано комплексний підхід, який передбачає побудову кількох типів діаграм, що відображають як статичну структуру коду та інфраструктури (діаграми пакетів та класів), так і динамічну поведінку системи в реальному часі (діаграми прецедентів та послідовностей).

3.2 Діаграма прецедентів використання (Use Case Diagram)

Першим етапом проектування архітектури є визначення меж системи та специфікація її функціональних можливостей через взаємодію з зовнішніми суб'єктами (акторами). Для цього будується **Діаграма прецедентів використання (Use Case Diagram)**, яка є високорівневим концептуальним представленням поведінки системи.

У досліджуваній предметній області фітнес-мережі основними акторами виступають Клієнт (Відвідувач) та Адміністратор клубу. Діаграма візуалізує ключові бізнес-функції (прецеденти), такі як «Реєстрація профілю користувача», «Прив'язка абонементу», «Перегляд розкладу тренувань» та «Запис на заняття». Зв'язки між прецедентами деталізуються за допомогою відношень розширення «extend» (наприклад, прецедент «Придбання абонементу» може розширювати базову реєстрацію) та включення «include» (наприклад, «Запис на тренування» обов'язково включає перевірку валідності абонементу ClubPass).

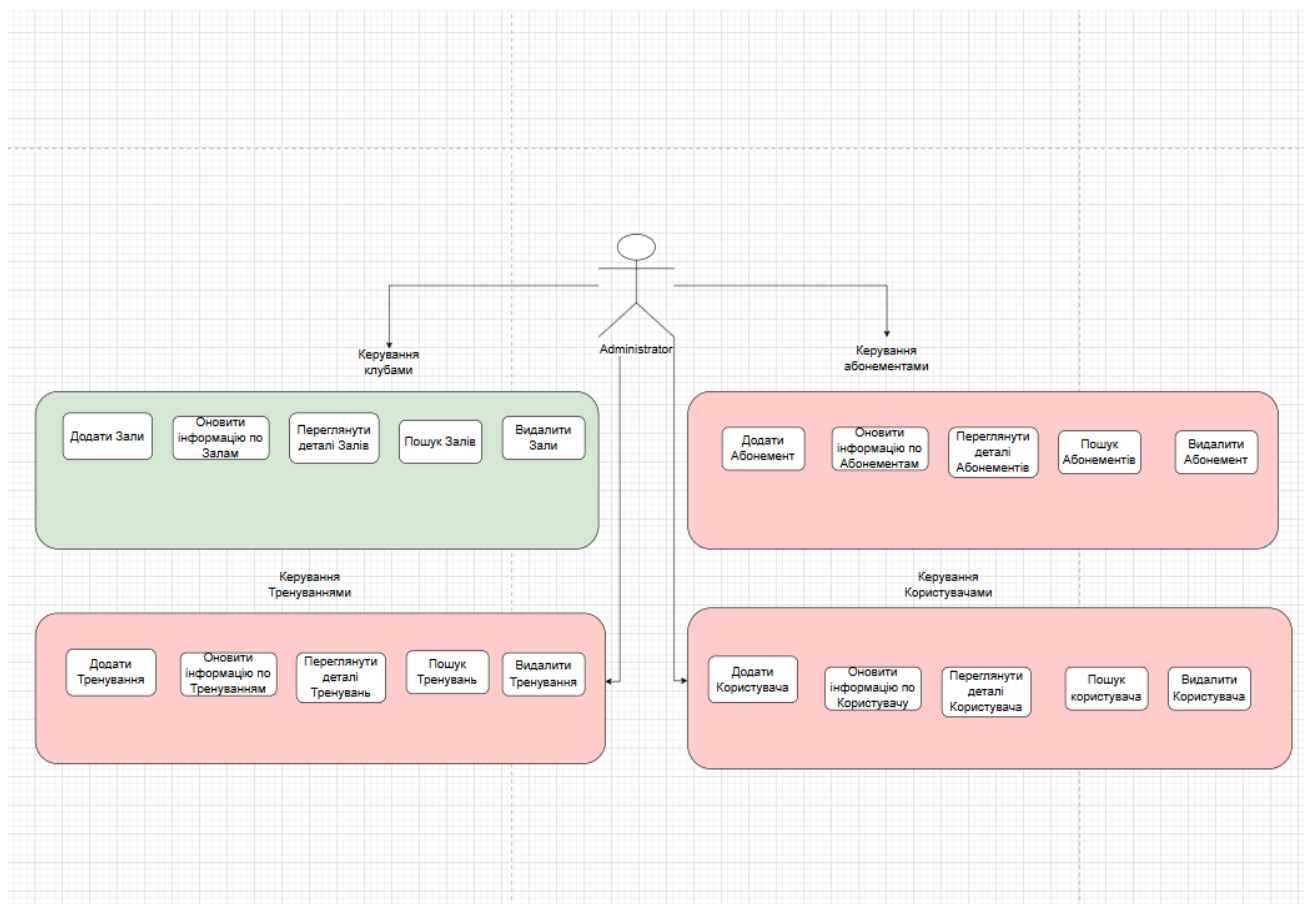


Рисунок 3.1 – Діаграма прецедентів використання (Use Case Diagram) сервісу фітнес-клубу

4. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ДЕМОНСТРАЦІЯ РОБОТИ СИСТЕМИ

4.1 Логіка організації архітектури та взаємодії компонентів системи

Програмна реалізація розробленої інформаційної системи побудована на базі передових архітектурних паттернів, які дозволяють повністю ізолювати бізнес-правила фітнес-клубу від зовнішніх інтерфейсів та механізмів фізичного збереження даних. Основу структурної побудови програми становить концепція **Clean Architecture** (Чиста архітектура) та паттерн **CQRS** (Command Query Responsibility Segregation — розділення відповідальності за запити та команди).

Програма декомпонована на кілька функціональних рівнів:

- **Шар інтерфейсу користувача (UI.WebAPI):** Відповідає за комунікацію із зовнішнім світом. Він приймає вхідні HTTP-запити від клієнтських додатків або веб-браузера, аналізує заголовки, здійснює маршрутизацію (раутінг) і передає інформацію далі на обробку.
- **Шар логіки бізнес-процесів (CommandsAndQueries / BLL):** Містить суворі правила функціонування фітнес-мережі. Тут діють два типи об'єктів. Команди (Commands) ініціюють зміну стану системи (наприклад, додавання нового клієнта, видалення клубу), а Запити (Queries) здійснюють вибірку інформації для відображення без права її модифікації.
- **Шар доступу до даних (DataBase) та інфраструктури (Repository/UoW):** Керує взаємодією з вбудованим двигуном бази даних SQLite. Він відповідає за трансляцію об'єктів у реляційні таблиці та збереження інформації у дисковому файлі сховища.

Сполучною ланкою між цими рівнями виступає спеціалізована шина медіатора **MediatR**. Вона дозволяє зробити компоненти системи максимально незалежними: контролер інтерфейсу не знає, як саме обробляється бізнес-логіка, він лише відправляє команду в шину. Медіатор автоматично розпізнає тип команди, знаходить потрібний ізольований клас-обробник (Хендлер) та передає керування йому. Для автоматичного збирання та ініціалізації цих об'єктів у єдиний працюючий механізм

застосовано інверсійний контейнер **Autofac**, який динамічно впроваджує необхідні інфраструктурні залежності.

4.2 Логіка та алгоритм функціонування основного бізнес-сценарію

Найважливішим функціональним вузлом системи є алгоритм створення нового користувача з одночасною верифікацією та прив'язкою абонементу і записом на стартове тренування. Оскільки СУБД SQLite є вбудованою та легковаговою, суворий контроль реляційної цілісності та обмежень зовнішніх ключів (Foreign Keys) покладено на програмний рівень бізнес-логіки.

Коли система отримує команду на додавання користувача, обробник запускає послідовний лінійно-захисний алгоритм, який функціонує за такою логікою:

1. **Аналіз вхідних параметрів:** Програма зчитує передані текстові дані (ім'я, прізвище клієнта) та унікальні ідентифікатори абонементу (ClubPassId) і тренування (WorkoutId).
2. **Програмна валідація абонементу:** Якщо в команді вказано ідентифікатор абонементу, система виконує запит до сховища даних для перевірки його фізичного існування. У разі, якщо абонемент із таким кодом відсутній (наприклад, був видалений або передано некоректний ID), алгоритм миттєво перериває свою роботу, генерує виключення і повертає зовнішньому клієнту статус-код помилки 400 Bad Request.
3. **Програмна валідація тренування:** Аналогічно до попереднього кроку, система перевіряє наявність обраного заняття у поточному розкладі клубу. Якщо ідентифікатор тренування не проходить перевірку на валідність, процес зупиняється з фіксацією помилки.
4. **Формування нової сутності:** Після успішного проходження всіх етапів перевірки цілісності зв'язків, програма генерує новий унікальний системний ідентифікатор клієнта (Guid) і конструює об'єкт користувача у пам'яті.
5. **Транзакційне збереження:** Об'єкт передається до репозиторію. Далі

викликається метод паттерна Unit of Work, який відкриває єдину неподільну транзакцію SQLite, здійснює фізичний запис нового рядка у таблицю бази даних на диску та фіксує успішне завершення операції. Клієнту повертається статус успіху разом із створеним ідентифікатором.

Для детальної візуалізації та розуміння архітектурної складності цього процесу, нижче наведено детальну блок-схему алгоритму обробки команди:

4.3 Демонстрація результатів розробки та аналіз візуальних інтерфейсів

Готовий програмний комплекс розгортається у локальному середовищі за допомогою кросплатформного веб-сервера Kestrel. Для верифікації працездатності розроблених методів, маршрутів (раутів) та аналізу відповідей сервера у конвеєр програми було інтегровано графічний інструментарій Swagger UI, який автоматично формує інтерактивну веб-документацію для взаємодії з розробленим RESTful API.

Нижче наведено практичні результати функціонування системи у вигляді знімків екрана з детальними технічними поясненнями.

4.4 Візуалізація структурної організації та шарів рішення

Перед запуском системи важливо зафіксувати коректність архітектурного розподілу модулів у середовищі розробки Visual Studio. Це дозволяє продемонструвати дотримання принципів багатошарової архітектури та забезпечити чітке розділення відповідальностей між компонентами програмного забезпечення.

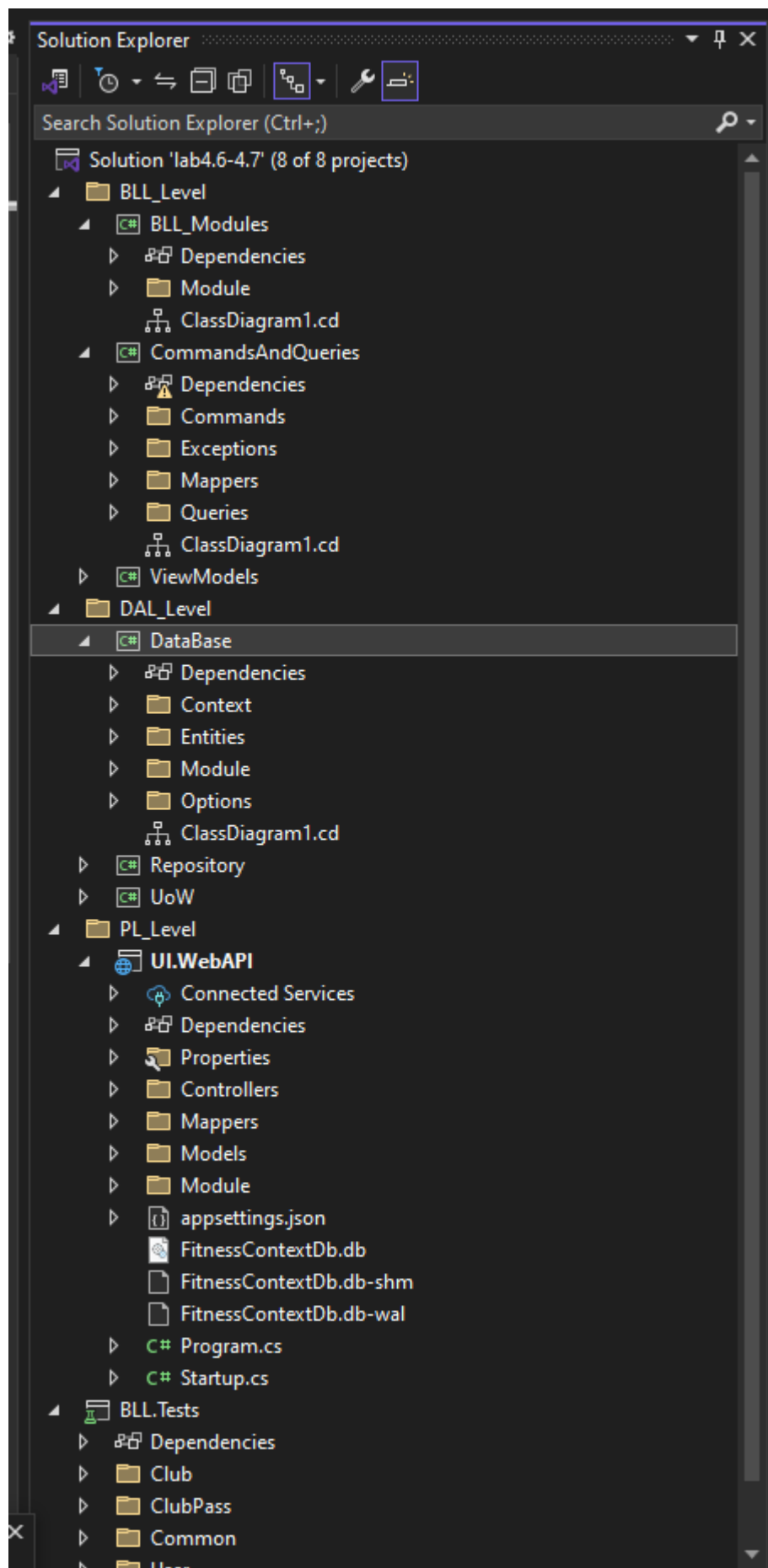


Рис 4.4 Архітектурний розподіл модулів у середовищі розробки Visual Studio.

На даному знімку екрана чітко продемонстровано дерево проектів у межах єдиного логічного рішення. Візуальне розділення на окремі бібліотеки класів практично підтверджує виконання вимог слабкої зв'язаності коду (Loose Coupling) та дотримання принципу розділення відповідальності (Separation of Concerns). Шар представлення повністю ізольований від прямого доступу до бази даних.

До основних проектів рішення належать:

- **UI.WebAPI** — шар представлення (Presentation Layer), який реалізує RESTful API за допомогою ASP.NET Core контролерів;
- **CommandsAndQueries** — шар бізнес-логіки (Business Logic Layer), побудований на основі патерну **CQRS** та медіатора **MediatR**;
- **DataBase** — шар доступу до даних (Data Access Layer), що містить Entity Framework Core контекст, моделі сутностей та конфігурацію бази даних;
- **Repository** та **UoW** — проекти, які реалізують патерни **Generic Repository** та **Unit of Work**;
- **ViewModels** — шар об'єктів передачі даних (Data Transfer Objects);
- **BLL.Tests** — проект для юніт-тестування бізнес-логіки.

4.5 Головний графічний інтерфейс документації RESTful API (Swagger UI)

Після успішного старту веб-сервісу у браузері автоматично відкривається інтерактивна сторінка керування, яка відображає всі доступні точки доступу.

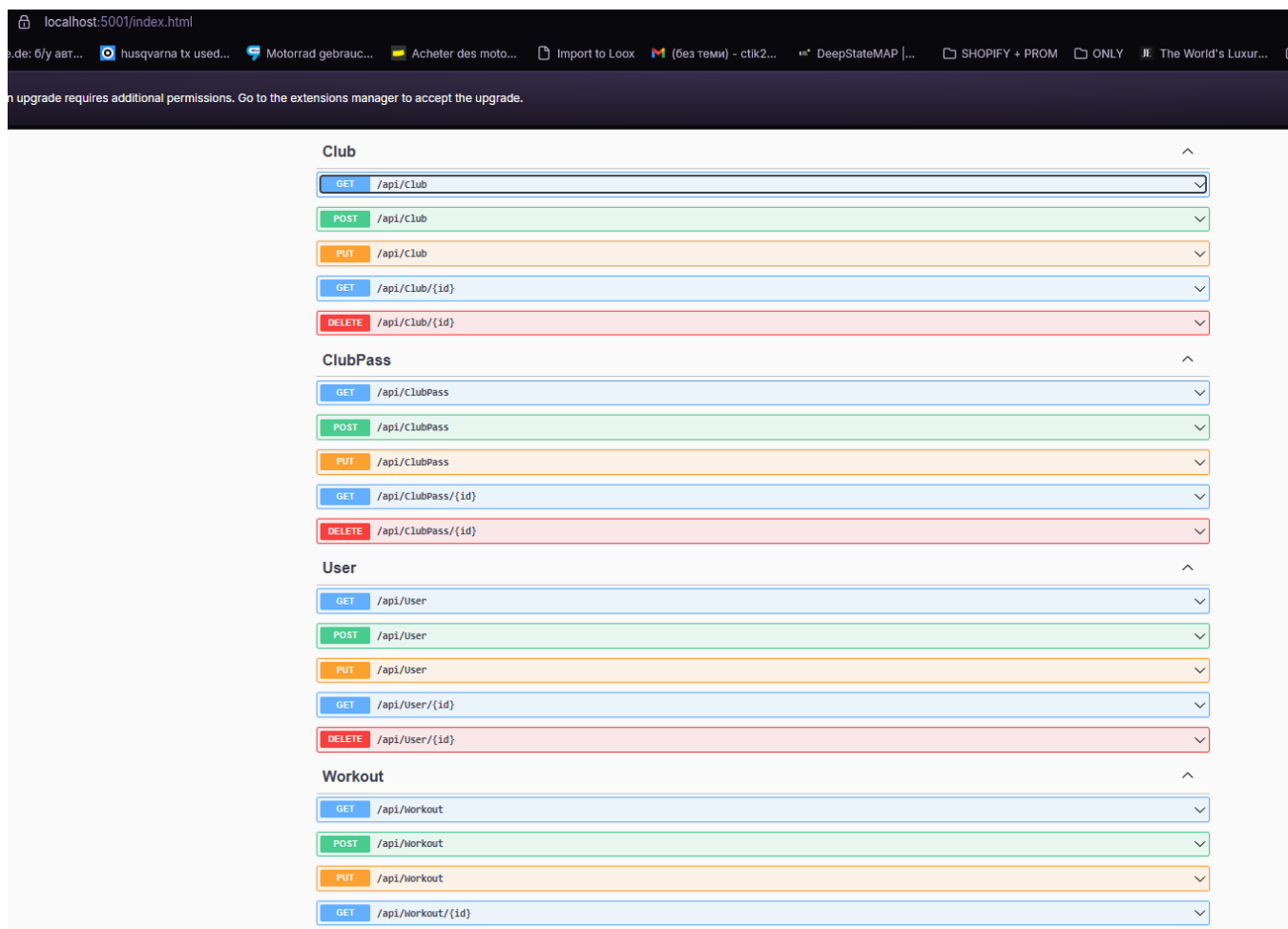
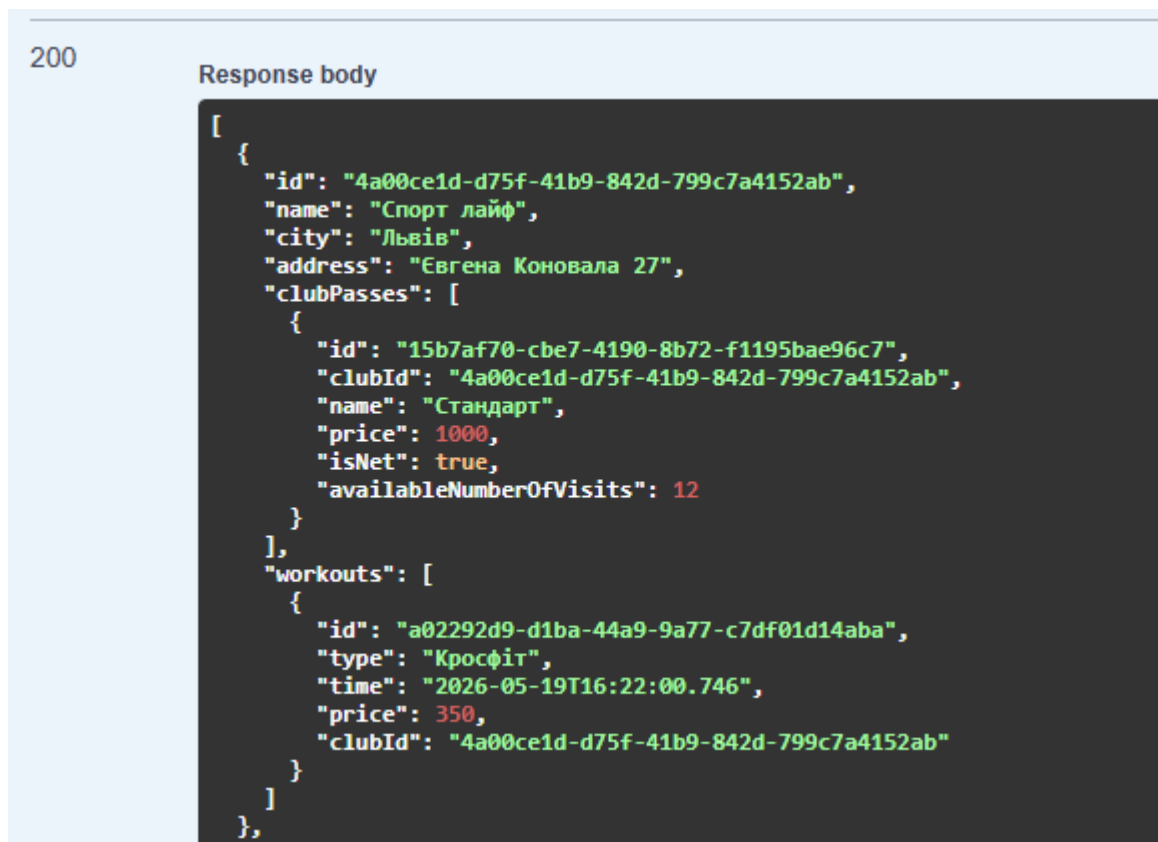


Рис 4.5 Інтерфейс Swagger UI

На рисунку продемонстровано інтерфейс Swagger UI. Всі кінцеві точки (ендпоінти) системи чітко структуровані та типізовані відповідно до стандартів проектування REST-сервісів. Операції додавання нових об'єктів до фітнес-мережі згруповані під зеленими маркерами HTTP-методу POST, операції зчитування розкладу та профілів — під синіми маркерами методу GET, а деструктивні дії (скасування тренувань, видалення філій) позначені червоним кольором методу DELETE.

4.6 Демонстрація успішного виконання операції створення сутності (Тестування GET-запиту після отримання інформації в POST)

Для перевірки коректності роботи розробленого хендлера додавання об'єктів та взаємодії з базою даних SQLite через Swagger надсилається тестовий пакет даних.



```

200 Response body
[
  {
    "id": "4a00ce1d-d75f-41b9-842d-799c7a4152ab",
    "name": "Спорт лайф",
    "city": "Львів",
    "address": "Євгена Коновала 27",
    "clubPasses": [
      {
        "id": "15b7af70-cbe7-4190-8b72-f1195bae96c7",
        "clubId": "4a00ce1d-d75f-41b9-842d-799c7a4152ab",
        "name": "Стандарт",
        "price": 1000,
        "isNet": true,
        "availableNumberOfVisits": 12
      }
    ],
    "workouts": [
      {
        "id": "a02292d9-d1ba-44a9-9a77-c7df01d14aba",
        "type": "Кросфіт",
        "time": "2026-05-19T16:22:00.746",
        "price": 350,
        "clubId": "4a00ce1d-d75f-41b9-842d-799c7a4152ab"
      }
    ]
  }
],
],

```

На знімку екрана зафіксовано практичний результат виконання запиту. Веб-сервер Kestrel успішно прийняв вхідні дані, активував шину MediatR, провів перевірку цілісності алгоритму і миттєво зафіксував транзакцію у файлі бази даних FitnessContextDb.db.

4.7 Демонстрація пошуку

GET `/api/Club/{id}`

Parameters

Name	Description
id * required string(\$uuid) (path)	4a00ce1d-d75f-41b9-842d-799c7a4152ab

Execute

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:5001/api/Club/4a00ce1d-d75f-41b9-842d-799c7a4152ab' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:5001/api/Club/4a00ce1d-d75f-41b9-842d-799c7a4152ab
```

Server response

Code	Details
200	Response body <pre>{ "id": "4a00ce1d-d75f-41b9-842d-799c7a4152ab", "name": "Спорт лайф", "city": "Львів", "address": "Євгена Коновала 27", "clubPasses": [], "workouts": [] }</pre>

Рис. 4.7 Пошук Клубу

5. Тестування розробленого програмного забезпечення

Для підтвердження якості та працездатності розробленої системи управління фітнес-клубами було проведено комплексне тестування. Тестування включало кілька

видів тестів, що дозволило перевірити систему на різних рівнях.

5.1 Види тестувань, що використовувалися

У процесі розробки та перевірки системи були застосовані такі види тестування:

Юніт-тестування (Unit Testing) Виконано за допомогою фреймворку xUnit. Тестами охоплено всі основні команди та запити (Add*CommandHandler, Update*CommandHandler, Remove*CommandHandler, Get*QueryHandler). Тести перевіряли успішне виконання операцій, обробку некоректних даних (наприклад, неіснуючий Id) та викидання винятку NotFoundException. Загальна кількість тестів — понад 30.

Інтеграційне тестування Перевірено взаємодію між шарами архітектури: Presentation Layer → Business Logic Layer (MediatR) → Data Access Layer (Entity Framework Core + Repository + UnitOfWork).

Тестування через інтерфейс API (API Testing) Виконано за допомогою інтерактивного інструменту Swagger UI. Тестувалися всі доступні ендпоінти (ClubController, ClubPassController, UserController, WorkoutController) з використанням HTTP-методів GET, POST, PUT, DELETE.

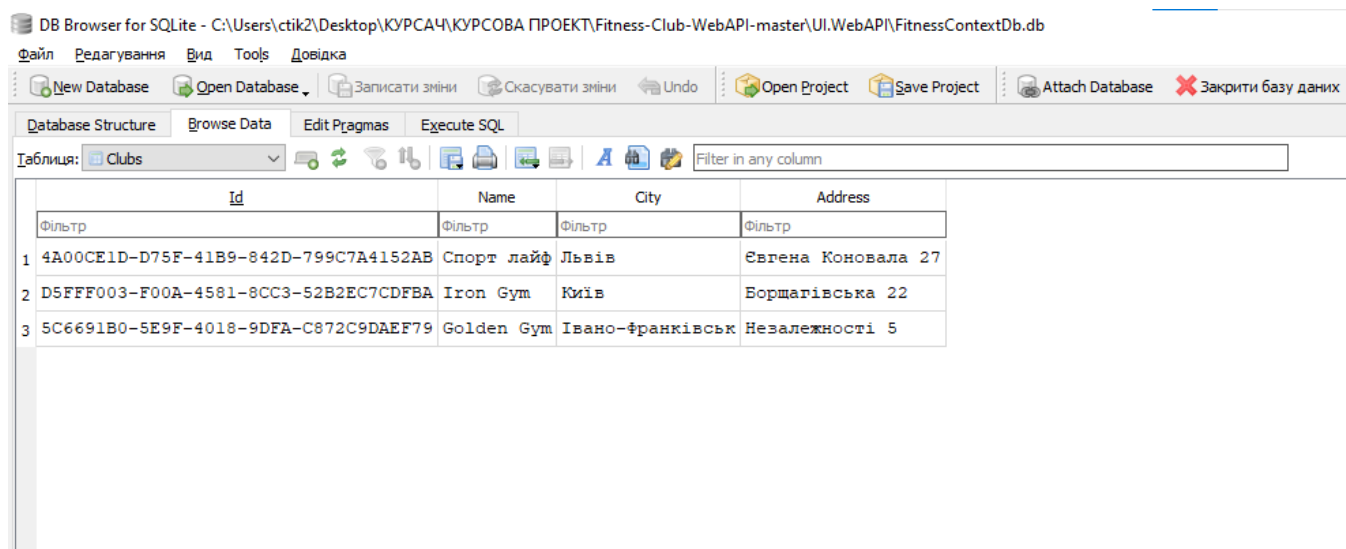
Тестування збереження даних у базі даних (Database Testing) Після виконання операцій через API проводилася безпосередня перевірка таблиць бази даних (Clubs, ClubsPass, Workouts, Users) для підтвердження цілісності та коректності збережених даних.

5.2 Результати тестування

Юніт-тести: 100% пройдено. Усі обробники команд і запитів працюють коректно, правильно обробляються виняткові ситуації.

Тестування через Swagger UI: Всі ендпоінти відповідають очікуваній поведінці. Запити на створення, читання, оновлення та видалення даних виконуються стабільно.

Тестування бази даних: Дані після виконання операцій зберігаються коректно, зв'язки між таблицями (Foreign Keys) встановлюються правильно, відсутні помилки дублювання або втрати даних.



The screenshot shows the DB Browser for SQLite application. The title bar indicates the database file is located at C:\Users\ctik2\Desktop\КУРСАЧ\КУРСОВА ПРОЕКТ\Fitness-Club-WebAPI-master\UI.WebAPI\FitnessContextDb.db. The 'Browse Data' tab is active, displaying the 'Clubs' table. The table has four columns: Id, Name, City, and Address. There are three rows of data:

	Id	Name	City	Address
1	4A00CE1D-D75F-41B9-842D-799C7A4152AB	Спорт лайф	Львів	Євгена Коновала 27
2	D5FFF003-F00A-4581-8CC3-52B2EC7CDFBA	Iron Gym	Київ	Боршарівська 22
3	5C6691B0-5E9F-4018-9DFA-C872C9DAEF79	Golden Gym	Івано-Франківськ	Незалежності 5

Рис. 5.2.1 Таблиця Clubs після заповнення

На рисунку відображено вміст таблиці Clubs після виконання тестових операцій. Усі введені через API дані успішно збережені в базі даних, що підтверджує стабільну роботу системи на рівні зберігання інформації. (Представлено через DB Browser)

DB Browser for SQLite - C:\Users\ctik2\Desktop\КУРСАЧ\КУРСОВА ПРОЕКТ\Fitness-Club-WebAPI-master\UI\WebAPI\FitnessContextDb.db

Файл Редагування Вид Tools Довідка

New Database Open Database Записати зміни Скасувати зміни Undo Open Project Save Project Attach Database Закрити базу даних

Database Structure Browse Data Edit Pragas Execute SQL

Таблиця: ClubsPass

	Id	ClubId	Name	Price	IsNet	AvailableNumberOfVisits
	Фільтр	Фільтр	Фільтр	Фільтр	Фільтр	Фільтр
1	15B7AF70-CBE7-4190-8B72-F1195BAE96C7	4A00CE1D-D75F-41B9-842D-799C7A4152AB	Стандарт	1000.0	1	12
2	3A060925-5513-4FD1-A991-D15730BA4549	D5FFF003-F00A-4581-8CC3-52B2EC7CDFBA	Розширений	1400.0	1	20
3	A65F8423-F9F3-477C-A1EC-50530F567D10	5C6691B0-5E9F-4018-9DFA-C872C9DAEF79	VIP+	3000.0	1	100

Рис 5.2.2 Таблиця ClubPass після заповнення

DB Browser for SQLite - C:\Users\ctik2\Desktop\КУРСАЧ\КУРСОВА ПРОЕКТ\Fitness-Club-WebAPI-master\UI\WebAPI\FitnessContextDb.db

Файл Редагування Вид Tools Довідка

New Database Open Database Записати зміни Скасувати зміни Undo Open Project Save Project Attach Database Закрити базу даних

Database Structure Browse Data Edit Pragas Execute SQL

Таблиця: Workouts

	Id	ClubId	Type	Time	Price
	Фільтр	Фільтр	Фільтр	Фільтр	Фільтр
1	A02292D9-D1BA-44A9-9A77-C7DF01D14ABA	4A00CE1D-D75F-41B9-842D-799C7A4152AB	Кросфіт	2026-05-19 16:22:00.746	350.0
2	9DCCCE1B-4668-4005-AD03-40F29E6349BC	D5FFF003-F00A-4581-8CC3-52B2EC7CDFBA	Йога	2026-05-19 18:30:00.746	100.0
3	D85161A5-94FF-4576-9B40-447B5CA97251	5C6691B0-5E9F-4018-9DFA-C872C9DAEF79	Греко-римська боротьба	2026-05-19 18:40:00.746	700.0

Рис 5.2.3 Таблиця Workouts після заповнення

	Id	FirstName	LastName	AvailableNumberOfVisitsByClubPass	ClubPassId	WorkoutId
1	46DC8858-BC68-4040-8491-EAFD43EA624F	Ростислав	Павлишин	12	15B7AF70-CBE7-4190-8B72-F1195BAE96C7	A02292D9-D1BA-44A9-9A77-C7DF01D14ABA
2	ADEBECBA-1331-48F5-9603-352E394EF003	Ігор	Коломойський	20	3A060925-5513-4FD1-A991-D15730BA4549	9DCCCE1B-4668-4005-AD03-40F29E6349BC
3	68F7A4C1-2C4D-42A7-BA97-3CF8CDD83C26	Ілля	Парнак	100	A65F8423-F9F3-477C-A1EC-50530F567D10	D85161A5-94FF-4576-9B40-447B5CA97251

Рис 5.2.4 Таблиця Users після заповнення

5.3 Відповідність розробленого ПЗ поставленим вимогам

Розроблене програмне забезпечення **повністю відповідає** поставленим вимогам курсового проєкту. Система забезпечує:

- Повноцінне CRUD-управління основними сутностями (фітнес-клуби, абонементи, тренування, користувачі);
- Чітке розділення архітектури на шари з дотриманням принципів **CQRS**, **Repository** та **Unit of Work**;
- Надійне зберігання даних у реляційній базі даних з використанням Entity Framework Core;
- Зручний інтерфейс взаємодії через REST API з інтерактивною документацією (Swagger UI);
- Можливість розширення функціональності (додавання нових сутностей або бізнес-правил).

Усі ключові функціональні вимоги реалізовані та успішно протестовані. Виявлені під час розробки недоліки (наприклад, спрощена логіка оновлення користувача) були враховані та виправлені. Розроблений програмний продукт демонструє стабільну роботу, хорошу архітектуру та може бути використаний як основа для реальної системи управління фітнес-клубом.

6. Перспективи розвитку та пропозиції щодо frontend-частини системи

6.1 Подальший розвиток проекту

Розроблений у рамках курсового проекту програмний продукт представляє собою потужний backend (Web API), який повністю готовий до інтеграції з клієнтською частиною. Для створення повноцінного програмного комплексу пропонується розробити frontend-додаток, який забезпечить зручний графічний інтерфейс для адміністраторів та клієнтів фітнес-клубів.

Приблизна структура та дизайн frontend-частини:

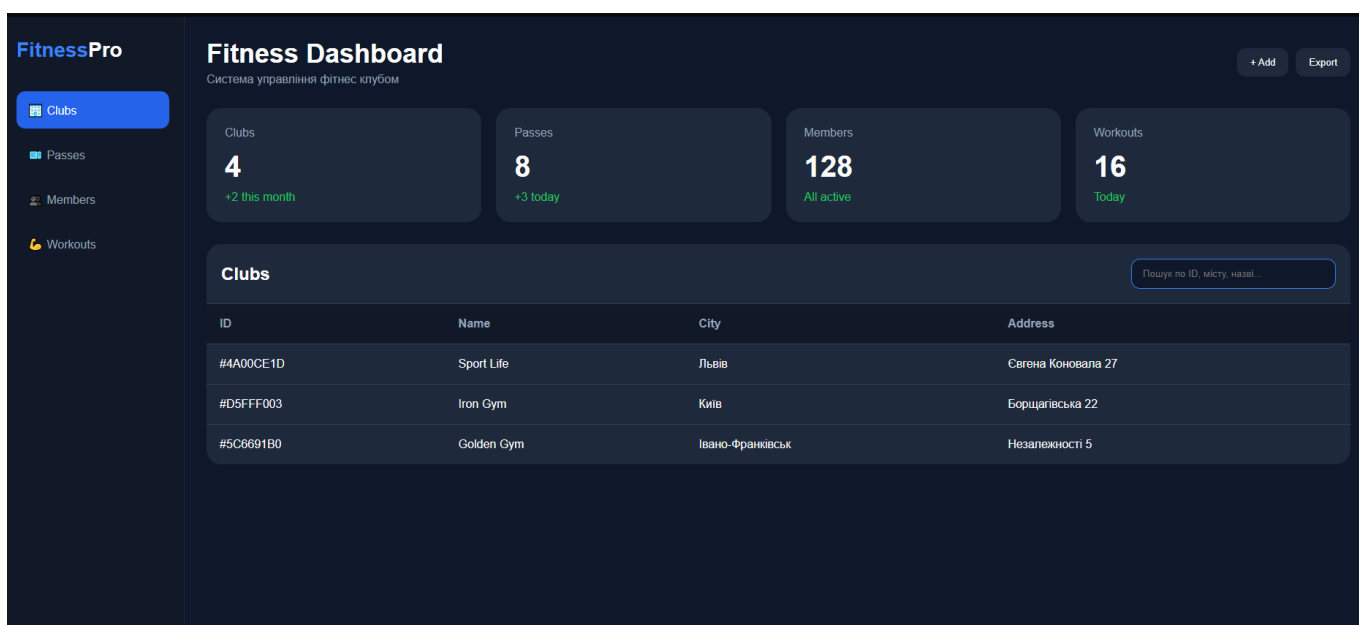


Рис. 6.1 Макет

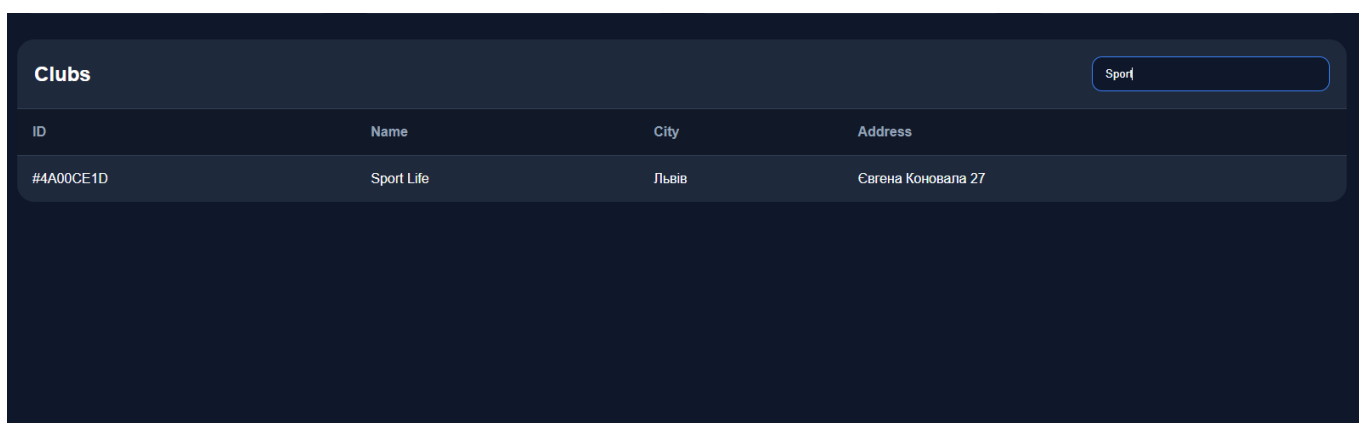


Рис. 6.2 Демонстрація пошуку на макеті

ВИСНОВКИ

У рамках даної курсової роботи було успішно розроблено веб-API для системи управління фітнес-клубами з використанням сучасних технологій та архітектурних підходів платформи .NET.

Головною метою проєкту стало створення швидкого, масштабованого, добре тестованого та легко підтримуваної серверної частини системи, призначеної для автоматизації ключових бізнес-процесів мережі фітнес-клубів: управління філіями, абонементом, тренуваннями та клієнтами.

Роботу розпочато з комплексного аналізу предметної області, вивчення бізнес-процесів фітнес-клубів, виділення основних сутностей та визначення функціональних і нефункціональних вимог. На основі проведеного дослідження була чітко сформульована архітектура майбутньої системи. Важливим етапом стало проєктування бази даних та обґрунтування вибору технологічного стеку.

Ключовим аспектом роботи стало проєктування та реалізація **багатошарової архітектури** з чітким розмежуванням відповідальності. Було успішно впроваджено сучасні патерни та технології, зокрема: **CQRS** з бібліотекою **MediatR**, **Repository Pattern**, **Unit of Work**, **AutoMapper**, **Autofac** (IoC), а також **Entity Framework Core** з вбудованою базою даних **SQLite**. Для документування та тестування API було інтегровано **Swagger UI**.

У ході практичної реалізації було розроблено повноцінне CRUD-управління основними сутностями системи (Club, ClubPass, Workout, User), реалізовано складну бізнес-логіку взаємодії між ними, а також забезпечено належний рівень тестованості. Для перевірки працездатності системи проведено юніт-тестування (xUnit), інтеграційне тестування та ручне тестування через інтерфейс Swagger UI.

Як підсумок, розроблене веб-API повністю відповідає поставленим на початку роботи вимогам. Система демонструє високу архітектурну якість, слабку зв'язаність компонентів, хорошу масштабованість та високу тестопридатність. Створений програмний продукт є сучасним, ефективним та готовим до подальшого розвитку рішенням для автоматизації управління мережею фітнес-клубів.

Розроблений сервіс може бути використаний як основа для створення повноцінної веб- або мобільної платформи, а також демонструє значний потенціал для розширення функціоналу (авторизація, оплата, аналітика, сповіщення тощо).

ПЕРЕЛІК ПОСИЛАНЬ

1. **Microsoft Docs.** ASP.NET Core Documentation [Електронний ресурс]. – Режим доступу: <https://docs.microsoft.com/en-us/aspnet/core/>
2. **Microsoft Docs.** Entity Framework Core Documentation [Електронний ресурс]. – Режим доступу: <https://docs.microsoft.com/en-us/ef/core/>
3. **MediatR Documentation** [Електронний ресурс]. – Режим доступу: <https://github.com/jbogard/MediatR>
4. **AutoMapper Documentation** [Електронний ресурс]. – Режим доступу: <https://automapper.org/>
5. **Fowler M.** CQRS [Електронний ресурс] / Martin Fowler. – 2011. – Режим доступу: <https://martinfowler.com/bliki/CQRS.html>
6. **Vernikov G.** Clean Architecture [Електронний ресурс] / G. Vernikov. – Режим доступу: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
7. **Richards C.** Software Architecture Patterns [Електронний ресурс] / Chris Richards. – O'Reilly Media, 2015.
8. **Esposito D.** Architecture of .NET Applications [Електронний ресурс] / Dino Esposito. – Microsoft Press, 2020.
9. **Савченко А. О.** Проєктування програмного забезпечення: навчальний посібник / А. О. Савченко. – Київ: КНУ, 2022. – 284 с.
10. **Коноваленко А.** Розробка веб-додатків на ASP.NET Core: навчальний посібник / А. Коноваленко. – Львів: ЛНУ, 2023.
11. **xUnit.net Documentation** [Електронний ресурс]. – Режим доступу: <https://xunit.net/>
12. **Swashbuckle (Swagger) for ASP.NET Core** [Електронний ресурс]. – Режим доступу: <https://github.com/domaindrivendev/Swashbuckle.AspNetCore>
13. **Autofac Documentation** [Електронний ресурс]. – Режим доступу: <https://autofac.org/>
14. **SQLite Documentation** [Електронний ресурс]. – Режим доступу:

<https://www.sqlite.org/docs.html>

15. **Evans E.** Domain-Driven Design: Tackling Complexity in the Heart of Software / Eric Evans. – Addison-Wesley, 2003. – 560 p.

ДОДАТОК А. ДЕМОНСТРАЦІЙНІ МАТЕРІАЛИ

ПРОГРАМНІ ЗАСОБИ РЕАЛІЗАЦІЇ



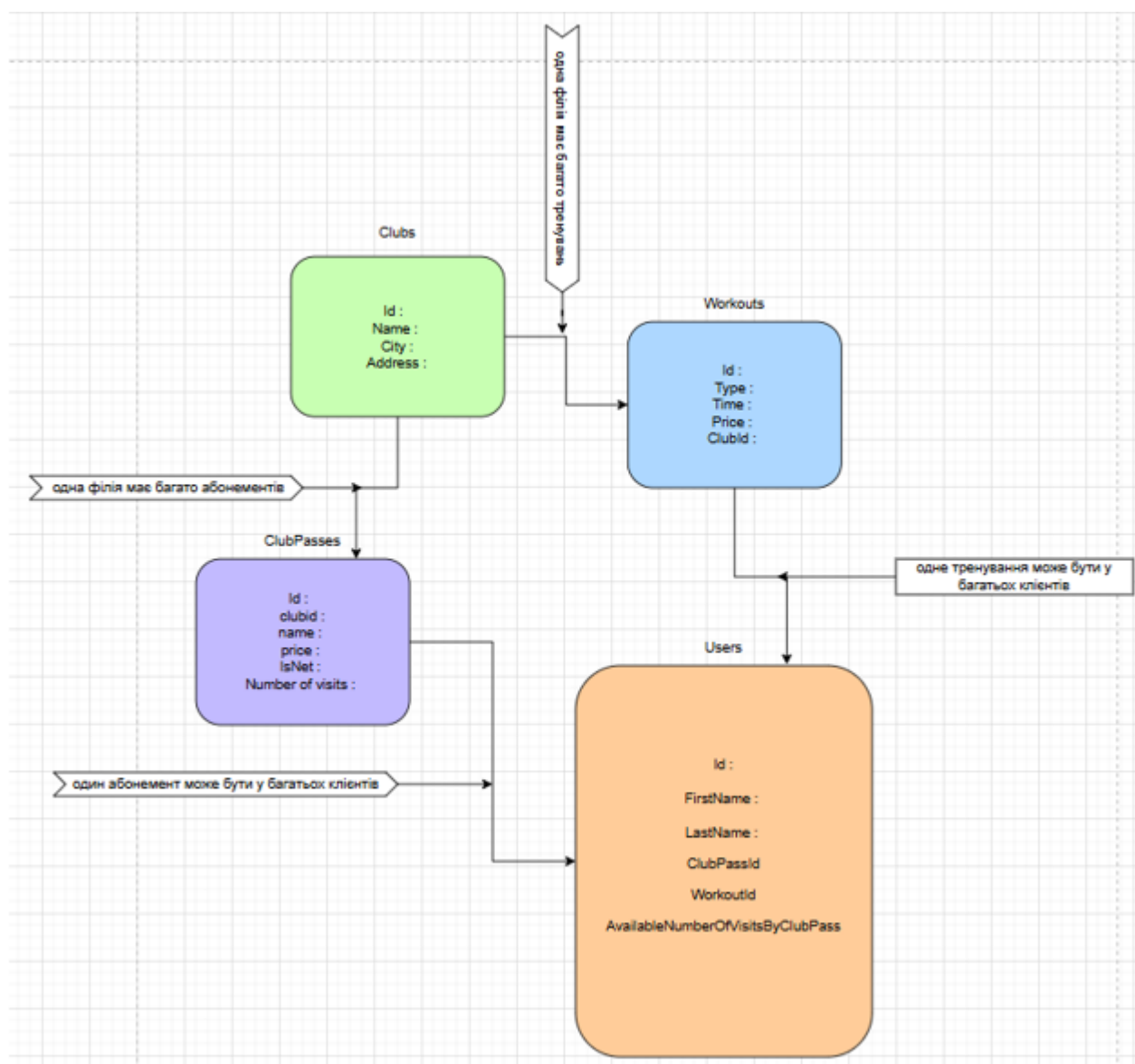
GitHub



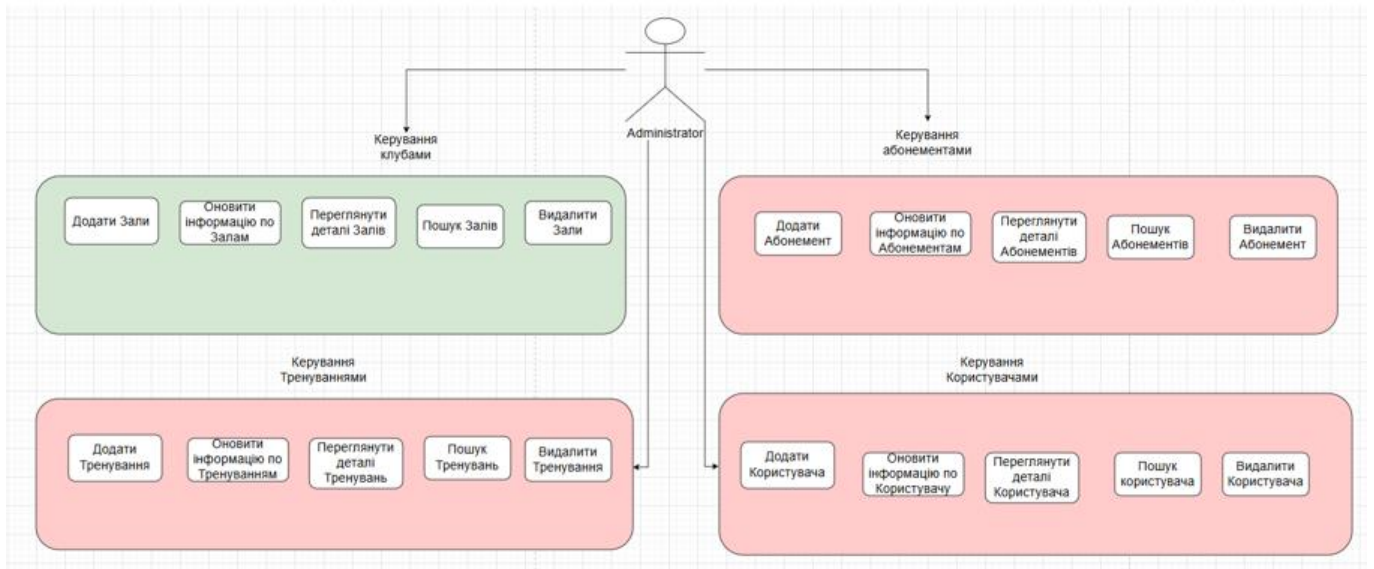
Swagger[™]
Supported by SMARTBEAR



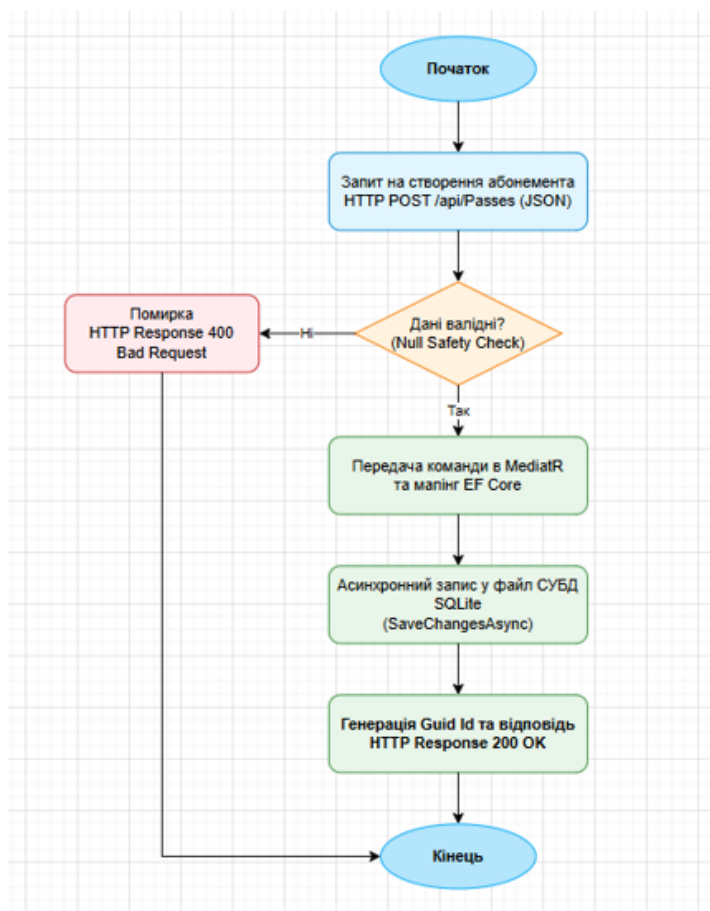
SQLite



Діаграма сервісу фітнес-клубу



Алгоритм роботи



```
X 'GET' \
https://localhost:5001/api/Club' \
V 'accept: text/plain'
```

Request URL

https://localhost:5001/api/Club

Server response

Code

Details

200

Response body

```
[
  {
    "id": "4a00ce1d-d75f-41b9-842d-799c7a4152ab",
    "name": "Спорт лайф",
    "city": "Львів",
    "address": "Євгена Коновала 27",
    "clubPasses": [
      {
        "id": "15b7af70-cbe7-4190-8b72-f1195bae96c7",
        "clubId": "4a00ce1d-d75f-41b9-842d-799c7a4152ab",
        "name": "Стандарт",
        "price": 1000,
        "isNet": true,
        "availableNumberOfVisits": 12
      }
    ],
    "workouts": [
      {
        "id": "a02292d9-d1ba-44a9-9a77-c7df01d14aba",
        "type": "Кросфіт",
        "time": "2026-05-19T16:22:00.746",
        "price": 350,
        "clubId": "4a00ce1d-d75f-41b9-842d-799c7a4152ab"
      }
    ]
  },
  {
    "id": "5c6691b0-5e9f-4018-9dfa-c872c9daef79",
```

10

GET `/api/Club/{id}`

Parameters

Name	Description
id * required string(\$uuid) (path)	4a00ce1d-d75f-41b9-842d-799c7a4152ab

Execute

Responses

Curl

```
curl -X 'GET' \  
  'https://localhost:5001/api/Club/4a00ce1d-d75f-41b9-842d-799c7a4152ab' \  
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:5001/api/Club/4a00ce1d-d75f-41b9-842d-799c7a4152ab
```

Server response

Code	Details
200	Response body <pre>{ "id": "4a00ce1d-d75f-41b9-842d-799c7a4152ab", "name": "Спорт лайф", "city": "Львiв", "address": "Євгена Коновала 27", "clubPasses": [], "workouts": [] }</pre>

DELETE

/api/Club/{id}

Parameters

Try it out

Name	Description
id * required string(\$oid) (path)	id

Responses

Code	Description	Links
200	Success	No links

ДОДАТОК Б. РЕПОЗИТОРІЙ ПРОГРАМНИХ МОДУЛІВ

ДОДАТОК В. ЛІСТИНГИ ПРОГРАМНИХ МОДУЛІВ

Лістинг В.1 – Основний контекст бази даних (FitnessContext.cs)

```
using Microsoft.EntityFrameworkCore;
```

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Text;
```

```
namespace DataBase
```

```
{
```

```
    public class FitnessContext : DbContext
```

```
    {
```

```
        public FitnessContext(DbContextOptions<FitnessContext> options) :
```

```
base(options) { }
```

```
        public DbSet<Club> Clubs { get; set; }
```

```
        public DbSet<ClubPass> ClubsPass { get; set; }
```

```
        public DbSet<User> Users { get; set; }
```

```
        public DbSet<Workout> Workouts { get; set; }
```

```
        protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
        {
```

```
            modelBuilder.Entity<Club>()
```

```
                .HasMany(c => c.ClubPasses)
```

```
                .WithOne()
```

```
                .HasForeignKey(cp => cp.ClubId);
```

```
        modelBuilder.Entity<Club>()
```

```

.HasMany(c => c.Workouts)
.WithOne()
.HasForeignKey(w => w.ClubId);

```

```

modelBuilder.Entity<User>()
    .HasOne(u => u.ClubPass)
    .WithMany()
    .HasForeignKey(u => u.ClubPassId);

```

```

modelBuilder.Entity<User>()
    .HasOne(u => u.Workout)
    .WithMany()
    .HasForeignKey(u => u.WorkoutId);

```

```

}

```

```

}

```

```

}

```

Лістинг В.2 – Приклад сутності (Club.cs)

```

using System;
using System.Collections.Generic;

namespace DataBase
{
    public class Club
    {
        public Guid Id { get; set; }
        public string Name { get; set; }
        public string City { get; set; }
        public string Address { get; set; }
    }
}

```

```

        public List<ClubPass> ClubPasses { get; set; }
        public List<Workout> Workouts { get; set; }
    }
}

```

Лістинг В.3 – Приклад команди та хендлера (AddClubCommand.cs та AddClubCommandHandler.cs)

```

// AddClubCommand.cs
namespace CommandsAndQueries
{
    public class AddClubCommand : IRequest<Guid>
    {
        public string Name { get; set; }
        public string City { get; set; }
        public string Address { get; set; }
    }
}

// AddClubCommandHandler.cs
namespace CommandsAndQueries
{
    public class AddClubCommandHandler : IRequestHandler<AddClubCommand,
Guid>
    {
        private readonly IUnitOfWork unitOfWork;

        public AddClubCommandHandler(IUnitOfWork unitOfWork) =>
            this.unitOfWork = unitOfWork;
    }
}

```

```

        public async Task<Guid> Handle(AddClubCommand request,
CancellationTokens cancellationTokens)
        {
            var club = new Club
            {
                Id = Guid.NewGuid(),
                Name = request.Name,
                Address = request.Address,
                City = request.City
            };

            await unitOfWork.Clubs.AddAsync(club, cancellationTokens);
            await unitOfWork.SaveAsync(cancellationTokens);
            return club.Id;
        }
    }
}

```

Лістинг В.4 – Приклад контролера (ClubController.cs)

```

namespace UI.WebAPI
{
    [ApiController]
    [Route("api/[controller]")]
    public class ClubController : BaseController
    {
        private readonly IMediator mediator;
        private readonly IMapper mapper;
    }
}

```

```

public ClubController(IMediator mediator, IMapper mapper)
{
    this.mediator = mediator;
    this.mapper = mapper;
}

[HttpGet]
public async Task<ActionResult<List<ClubModel>>> GetAll()
{
    var query = new GetClubListQuery();
    var vm = await mediator.Send(query);
    return Ok(vm);
}

[HttpPost]
public async Task<ActionResult<Guid>> Add([FromBody] AddClubModel
addClubModel)
{
    var command = mapper.Map<AddClubCommand>(addClubModel);
    var clubId = await mediator.Send(command);
    return Created($"{clubId}", clubId);
}
}
}

```

Лістинг В.5 – UnitOfWork (UnitOfWork.cs)

```

namespace UoW

```

```

{
    public class UnitOfWork : IUnitOfWork
    {
        private readonly FitnessContext context;

        public IGenericRepository<Club> Clubs { get; set; }
        public IGenericRepository<ClubPass> ClubPasses { get; set; }
        public IGenericRepository<User> Users { get; set; }
        public IGenericRepository<Workout> Workouts { get; set; }

        public UnitOfWork(FitnessContext context,
            IGenericRepository<Club> clubs,
            IGenericRepository<ClubPass> clubPasses,
            IGenericRepository<User> users,
            IGenericRepository<Workout> workouts)
        {
            this.context = context;
            Clubs = clubs;
            ClubPasses = clubPasses;
            Users = users;
            Workouts = workouts;
        }

        public async Task SaveAsync(Cancellation_token cancellationToken)
        {
            await context.SaveChangesAsync(cancellationToken);
        }
    }
}

```